UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE

Verifica Unità 2: soluzioni e griglia

Risposte corrette, punteggi e indicazioni di correzione. Totale 100 punti; voto in decimi = punti ÷ 10, arrotondato al mezzo voto (60 punti = 6). Nel codice si valuta la logica: piccoli errori di sintassi non tolgono punti.

1. Scelta multipla · 16 punti (2 ciascuna)

Dom.	Risposta corretta	Note
a	**deve scegliere tra coerenza e disponibilità**	Tranello: "sceglie due su tre". La partizione non si sceglie (lezione 1).
b	**non può sapere se il versamento è avvenuto**	Richiesta persa, server lento o risposta persa sono indistinguibili (lezione 2).
c	**ogni servizio possiede il proprio database**	Lezione 3.
d	`"2027-05-20"`	JSON non ha date: stringa ISO 8601. La D usa virgolette singole: non è JSON.
e	**in un elemento Fault dentro il Body**	Lezione 6.
f	**il sostituto locale che fa il marshalling e spedisce la chiamata**	La A è il registro (RMI), la B l'IDL o il WSDL.
g	`PUT /api/partite/4/risultato`	PUT è idempotente; aggiungere un goal con POST no.
h	**il browser, prima di passare la risposta allo script**	curl e Java non applicano il CORS (lezione 12).

2. Vero o falso · 10 punti (1 ciascuna)

N.	V/F	Motivo
1	**F**	Resta un solo server: un solo punto di guasto. Serve scalare in orizzontale.
2	**F**	RMI funziona solo tra programmi Java.
3	**V**	Il bus "intelligente e indispensabile" (lezione 3).
4	**V**	HTTP/3, 2022.
5	**V**	portType, binding, service.
6	**F**	Rimosso dal JDK 11: serve Jakarta XML Web Services (Metro).
7	**F**	È l'unico vincolo facoltativo.
8	**F**	Un 500 è una risposta: si controlla statusCode(). Le eccezioni sono per i problemi di rete.
9	**V**	Con -d curl passa a POST (con Content-Type x-www-form-urlencoded).
10	**F**	La Promise si risolve, con r.ok falso.

3. Scambio HTTP · 12 punti (2 ciascuna)

- a) **https://torneo.scuola.it/api/squadre/12** (schema dall'ipotesi HTTPS; accettato http://, o anche solo /api/squadre/12 con 1 punto).
- b) **201 Created indica che è stata creata una nuova risorsa, il cui URI è in Location**; 200 dice solo "ok".
- c) **Porta il token che identifica l'utente in ogni richiesta: il server non deve ricordare sessioni, vincolo stateless**. 1 punto per lo scopo, 1 per il vincolo.
- d) **POST non è idempotente: si crea una seconda squadra (id 13)**; accettato anche "il server risponde 409 se controlla i nomi doppi", purché si citi la non idempotenza.
- e) **401** (1 punto) · **400**, accettato 422 (1 punto).
- f) **La lunghezza del corpo della richiesta, in byte** (non caratteri). Solo "lunghezza": 1 punto.

4. SOAP e WSDL · 10 punti

- a) **Obbligatori Envelope e Body; facoltativo Header** (3 punti, 1 per elemento corretto).
- b) **POST; Content-Type text/xml** (SOAP 1.1; per 1.2 sarebbe application/soap+xml). Accettato anche l'header SOAPAction come dettaglio in più.
- c) `getClassificaResponse`.
- d) **Operazioni nel portType; indirizzo nel service (port, soap:address)**.
- e) `GET /api/gironi/A/classifica`; accettato `GET /api/classifica?girone=A`. Con POST: 0 punti.

5. Codici · 12 punti (2 ciascuno)

a) **204** (200 con corpo: 1 punto) · b) **409** · c) **401** · d) **403** (accettato 404 solo se motivato) · e) **405** · f) **406**.

6. Errori nel server Java · 10 punti (2,5 ciascuno, fino a 4)

- **Riga 1**: HashMap con un pool di thread → `ConcurrentHashMap`.
- **Riga 4**: squadra non trovata → codice **404**, non 200.
- **Riga 4–5**: manca `return` dopo la risposta d'errore: il codice prosegue e tenta una seconda risposta.
- **Riga 6**: senza corpo la lunghezza è **−1**, non 0 (0 significa chunked).
- **Riga 7**: l'header va impostato **prima** di sendResponseHeaders, altrimenti non viene inviato.

Gli errori sono cinque: ne bastano quattro per il punteggio pieno. Errore trovato senza correzione: 1 punto.

7. Che cosa stampa · 8 punti

- a) **D, poi A 404 false, poi B undefined**. 1 punto per D per prima (codice sincrono), 1 per A con 404 false, 2 per B undefined (la Promise non viene rifiutata, il JSON d'errore non ha "nome"). Chi scrive C invece di A e B: 1 punto al massimo.
- b) **D, poi C con il messaggio dell'errore di rete** (in Chrome "Failed to fetch").
- c) **Dopo la fetch:** `if (!r.ok) throw new Error("HTTP " + r.status);` così il catch stampa "C HTTP 404".

8. Progetto · 12 punti

Metodo e URI (soluzione di riferimento)	Successo	Errore
`GET /api/squadre`	200	401 se l'API è protetta
`GET /api/partite/{id}`	200	404

`POST /api/partite`	201 + Location	400, 403, 409 (squadra già impegnata)
`PUT /api/partite/{id}/risultato`	200 o 204	400, 403, 404
`DELETE /api/partite/{id}`	204	404, 403
`GET /api/gironi/{g}/classifica`	200	404

a) 1,5 punti per endpoint: 0,5 metodo, 0,5 URI senza verbi, 0,5 codici coerenti. Accettati PATCH per il risultato e soluzioni equivalenti ben motivate. b) **Origini diverse (host diverso): il server deve rispondere al preflight OPTIONS (204) con Access-Control-Allow-Origin: https://www.scuola.it, Access-Control-Allow-Methods con POST, Access-Control-Allow-Headers: Content-Type (e Authorization se usato); gli stessi header di Allow-Origin anche sulle risposte vere**. 1 punto per il preflight, 2 per gli header.

9. Casi di test · 4 punti (2 ciascuno)

Esempi validi: **PUT su una partita inesistente → 404**; **gol negativi o non numerici → 400**; **JSON non valido → 400**; **senza token → 401**; **token di uno studente → 403**; **PATCH o metodo non previsto → 405**. 1 punto per la richiesta, 1 per il codice atteso corretto.

10. Domanda aperta · 6 punti

Punti attesi (2 punti per aspetto corretto, fino a 6): SOAP ha un **contratto** WSDL obbligatorio e rigido, REST di solito OpenAPI facoltativo; SOAP usa sempre **XML** in una busta, REST rappresentazioni spesso JSON, più leggere; SOAP usa HTTP solo come **trasporto** (tutto POST), REST ne sfrutta metodi, codici e **cache**; SOAP ha standard WS-* per sicurezza e transazioni, utili nei sistemi aziendali; REST si prova con curl e si usa facilmente da browser e app. Conclusione attesa: tenerli entrambi è ragionevole (il gestionale esistente non va riscritto, le app nuove usano REST), magari con un gateway o un servizio che traduce. Penalizzare affermazioni false come "SOAP non è sicuro" o "REST è un protocollo".

Distribuzione degli argomenti: lezioni 1-3 (es. 1a-c, 1f, 2.1-3, 10), lezioni 4-5 (1d, 2.4, 3, 5), lezioni 6-7 (1e, 2.5-6, 4, 10), lezioni 8-9 (1g, 2.7, 3, 6, 8), lezione 10 (2.8), lezione 11 (2.9, 9), lezione 12 (1h, 2.10, 7, 8b).